

HTTP, CoAP, and MQTT

Working Principles, Protocol Internals, and Message Flows for IoT Applications

IoT Communication Protocols

Presented by: Dr. Anand D. Mane

1. Introduction

- Application-layer protocols determine how devices exchange data once lower layers (PHY/MAC, IPv6/6LoWPAN, routing) have delivered packets to the right place.
- HTTP, CoAP, and MQTT sit at this layer but were designed for very different operating conditions.
- HTTP: general-purpose, well-resourced web communication.
- CoAP: constrained devices needing a RESTful model over lossy links.
- MQTT: many-to-many telemetry distribution at scale.
- This document explains the internal working of each protocol , with wire-level message examples, and closes with a comparative analysis.

2.1 HTTP — Protocol Stack and Transport

- HTTP is a text-based, request-response protocol layered over TCP (connection-oriented, reliable transport).
- Application: HTTP
- Transport: TCP (port 80) / TLS+TCP for HTTPS (port 443)
- Network: IPv4 / IPv6
- Link/PHY: Ethernet, Wi-Fi, etc.
- TCP guarantees ordered, reliable delivery via a three-way handshake, sequence numbers, and retransmission — so HTTP inherits reliability rather than implementing it.
- This is precisely why HTTP is 'heavy' for constrained nodes: every request pays the cost of a TCP handshake (and, for HTTPS, a TLS handshake) plus verbose ASCII headers.

2.2 HTTP — Request–Response Model

- HTTP follows a strict client-initiated request/response exchange.
- The client (e.g., a mobile app or gateway) sends a request; the server (e.g., a device's embedded web server, or a cloud endpoint) returns exactly one response.

Method	Semantics	Typical IoT use
GET	Retrieve a resource's current representation	Read a sensor's current temperature value
POST	Submit data to be processed / create a resource	Upload a batch of sensor readings
PUT	Replace a resource's full state	Set a thermostat's target temperature
DELETE	Remove a resource	Delete a stored schedule/rule

2.3 HTTP — Working Example

- **Scenario: a mobile app reads the current temperature from a smart thermostat's REST API.**
- Step 1 — TCP handshake: SYN, SYN-ACK, ACK. Connection established (3 packets, 1.5 round trips) before any HTTP data is sent.
- Step 2 — (If HTTPS) TLS handshake: additional round trips for cipher negotiation and certificate exchange.
- Step 3 — Client sends: GET /api/v1/thermostat/temperature HTTP/1.1 (Host, Accept: application/json, Connection: keep-alive)
- Step 4 — Server responds: HTTP/1.1 200 OK, Content-Type: application/json, body {"tempC": 23.5, "unit": "C"}
- Step 5 — Connection kept alive (HTTP/1.1 default) or torn down with FIN/ACK, releasing TCP resources.
- Overhead for one reading: a TCP handshake, hundreds of bytes of ASCII headers, and optionally a TLS handshake — to move ~27 bytes of payload.

2.4 Where HTTP Fits in IoT Architectures

- HTTP is typically used at the boundary between the IoT network and the rest of the internet.
- Example: a gateway aggregates CoAP or MQTT traffic from constrained nodes and exposes it as a conventional REST API to cloud services and dashboards.
- HTTP is rarely used directly on the constrained nodes themselves.

3.1 CoAP — Protocol Stack and Design Goals

- CoAP (RFC 7252) brings a RESTful, HTTP-like interaction model to constrained nodes and lossy networks.
- Message format is compact and binary; transport is connectionless.
- Application: CoAP
- Transport: UDP (port 5683) / DTLS+UDP for secure CoAP (port 5684)
- Network: IPv6 (often via 6LoWPAN over IEEE 802.15.4)
- Link/PHY: IEEE 802.15.4, BLE, etc.
- Two-layer internal architecture: a Messaging layer (UDP, async delivery/deduplication/retransmission) and a Request/Response layer (GET/POST/PUT/DELETE) mapped onto a 4-byte binary header instead of verbose text.

3.2 CoAP — Message Types

Type	Meaning	Behaviour
CON (Confirmable)	'Please confirm you received this'	Retransmitted with exponential backoff until an ACK arrives or retries are exhausted
NON (Nonconfirmable)	Fire-and-forget	Sent once; no retransmission (e.g., repeated sensor readings where occasional loss is acceptable)
ACK (Acknowledgement)	Confirms a CON message	Carries the response piggybacked, or sent empty first if processing takes time
RST (Reset)	'I got your message but cannot process it'	Sent when a CON/NON message cannot be handled (e.g., unknown request)

3.3 CoAP — Message Header and Options

- Every CoAP message starts with a fixed 4-byte binary header: Version, Type, Token Length, Code, Message ID.
- Followed by an optional Token, a set of compact Options (encoded as delta + length, e.g. Uri-Path, Content-Format, Observe), and an optional Payload.
- This is dramatically smaller than an HTTP header block — typically 4–20 bytes of overhead versus hundreds of bytes for HTTP/TLS.

3.4 CoAP — Confirmable GET with Retransmission (1/2)

- **Scenario: a border-router gateway requests soil-moisture from a battery-powered sensor node over a lossy 802.15.4 mesh.**
- Step 1 — Client sends CON GET /sensors/moisture, MID=0x7A31, Token=0x9E
- Step 2 — The UDP datagram is lost in transit (no ACK, no reliability from the transport itself).
- Step 3 — CoAP's own reliability kicks in: client waits ACK_TIMEOUT (default ~2s, randomised) and, on timeout, retransmits the identical message with the same Message ID.

3.4 CoAP — Confirmable GET with Retransmission (2/2)

- Step 4 — Each retry doubles the timeout (exponential backoff), up to MAX_RETRANSMIT (default 4 retries).
- Step 5 — Retransmitted datagram arrives; sensor replies with ACK 2.05 Content, MID=0x7A31, Token=0x9E, Content-Format: application/json, Payload: {"moisture": 42}
- Step 6 — Client's deduplication cache recognises the Message ID and discards duplicate ACKs from delayed retransmissions.
- This shows CoAP re-implementing exactly the reliability HTTP gets 'for free' from TCP — but only when requested (CON), skipping it entirely for NON messages.

3.5 The Observe Option — Lightweight Publish/Subscribe

- CoAP extends the base request/response model with the Observe option (RFC 7641).
- A client sends a GET with Observe=0, and the server keeps the request 'open'.
- The server pushes a new notification (with an incrementing Observe sequence number) every time the resource's value changes — without the client having to poll.
- This gives CoAP a pub/sub-like capability while remaining strictly a two-party protocol (no broker), unlike MQTT.

3.6 CoAP–HTTP Mapping

- CoAP mirrors HTTP's method and response-code semantics.
- A gateway can translate transparently between the two, e.g., CoAP 2.05 Content maps to HTTP 200 OK, CoAP 4.04 Not Found maps to HTTP 404.
- This is what allows CoAP-based sensor networks to be exposed as ordinary REST APIs at the internet boundary.

4.1 MQTT — Architecture: Publish/Subscribe over a Broker

- Unlike HTTP and CoAP, MQTT is not a request/response protocol between two parties — it is a publish/subscribe protocol mediated by a central Broker.
- Publishers and Subscribers never talk to each other directly; they only talk to the broker, which decouples them completely in time, space, and synchronisation.
- Application: MQTT
- Transport: TCP (port 1883) / TLS+TCP for MQTTS (port 8883)
- Roles: Publisher, Subscriber, Broker
- This decoupling makes MQTT suitable for many-to-many telemetry: a hundred sensors can publish while a dashboard, logging service, and alerting service subscribe independently, with no publisher needing to know who is listening.

4.2 MQTT — Topics and Wildcards

- Messages are published to a Topic, a hierarchical, slash-separated string (e.g., building1/floor2/room5/temperature).
- Subscribers register interest using Topic Filters, which may include wildcards:

Wildcard	Meaning	Example filter matches
+	Single-level wildcard	building1/+/room5/temperature matches any floor
#	Multi-level wildcard (must be last)	building1/# matches everything under building1

4.3 MQTT — Quality of Service (QoS) Levels

- MQTT defines three QoS levels that trade reliability for overhead, chosen per-message by the publisher:

QoS	Guarantee	Handshake
0 — At most once	Fire-and-forget; message may be lost	PUBLISH only, no acknowledgement
1 — At least once	Message is delivered, possibly duplicated	PUBLISH → PUBACK
2 — Exactly once	Message delivered exactly once, no duplicates	PUBLISH → PUBREC → PUBREL → PUBCOMP (4-step handshake)

4.4 MQTT — Step-by-Step Working Example (1/2)

- **Scenario: a temperature sensor publishes readings at QoS 1; a dashboard subscribes to receive them.**
- Step 1 — Both publisher and subscriber open a TCP connection and send CONNECT (Client ID, Keep-Alive=60s, optional Last Will and Testament).
- Step 2 — Broker replies with CONNACK (Session Present flag + return code 0 = accepted) to both clients.
- Step 3 — Dashboard subscribes: SUBSCRIBE Topic=.../temperature QoS=1 → SUBACK GrantedQoS=1
- Step 4 — Sensor publishes: PUBLISH Topic=.../temperature Payload={"tempC":23.5} QoS=1

4.4 MQTT — Working Example (2/2)

- Step 5 — Broker stores/forwards the message to matching subscribers and returns PUBACK to the sensor ('at least once' — re-delivery possible, hence duplicates downstream).
- Step 6 — Broker forwards the payload to the dashboard as its own PUBLISH; dashboard responds with its own PUBACK.
- Step 7 — Keep-Alive: if no message passes within $1.5 \times \text{Keep-Alive}$, broker assumes ungraceful disconnect and publishes the Last Will message ("offline", Retained).
- Step 8 — Retained messages: if marked Retain=true, the broker immediately delivers the last known value to any newly-joining subscriber.

4.5 Why the Broker Model Matters

- Publishers and subscribers are fully decoupled.
- MQTT scales naturally to thousands of constrained devices publishing to a small, well-resourced broker.
- It tolerates devices joining/leaving at any time — a pattern that fits fleets of sensors far better than the strict two-party model of HTTP or CoAP.

5. Comparative Analysis (1/2)

Aspect	HTTP	CoAP	MQTT
Transport	TCP (connection-oriented)	UDP (connectionless)	TCP (connection-oriented)
Interaction model	Request/Response (2-party)	Request/Response (2-party), + Observe for push	Publish/Subscribe via broker (many-to-many)
Message format	Text (ASCII headers)	Binary, compact (4-byte header)	Binary, compact fixed header
Reliability	Inherited from TCP	Built into CoAP itself (CON/ACK, backoff)	Inherited from TCP + QoS 0/1/2 at app layer

5. Comparative Analysis (2/2)

Aspect	HTTP	CoAP	MQTT
Typical overhead	Hundreds of bytes/message	~4–20 bytes/message	~2–14 byte fixed header + variable
Security	TLS (HTTPS)	DTLS	TLS (MQTTS)
Best fit	Gateways, cloud APIs, dashboards	Constrained nodes, sensor/actuator REST	Many-to-many telemetry, fleets of devices

6. Summary

- HTTP, CoAP, and MQTT solve the same underlying problem — moving application data between endpoints — under three different constraint profiles.
- HTTP assumes abundant resources and a stable connection, leaning entirely on TCP for reliability.
- CoAP keeps HTTP's RESTful method/response semantics but strips the transport down to UDP, rebuilding only the reliability it needs (via CON/ACK and exponential backoff) — a natural fit for battery-powered, lossy-network sensor nodes.
- MQTT abandons the two-party request/response model in favour of a broker-mediated publish/subscribe architecture with tunable QoS, Retained messages, and Last-Will notifications — the protocol of choice wherever many devices must fan data out to many consumers reliably and asynchronously.